

Palindrome Checker and Sorting Algorithms

Fitsum Gelaye

October 4, 2024

1 Introduction

This document presents the analysis of a list of magic items which are being filtered to identify specific palindromes in every element of the list, if there is one. The document also presents the implementation of several sorting algorithms which are being used to sort the list after it has been shuffled into a random sequence. The ultimate purpose is to find which sorting mechanism has the best performance on average.

2 Stack Implementation (Stack.h)

Listing 1: Stack.h: Custom Stack Implementation

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 #include <iostream>
5 #include <vector>
6
7 template<typename T>
8 class Stack {
9 private:
10     std::vector<T> elements;
11
12 public:
13     void push(const T& item) {
14         elements.push_back(item);
15     }
16
17     void pop() {
18         if (!elements.empty()) {
19             elements.pop_back();
20         }
21     }
```

```

22
23     T top() const {
24         if (!elements.empty()) {
25             return elements.back();
26         }
27         throw std::out_of_range("Stack is empty");
28     }
29
30     bool empty() const {
31         return elements.empty();
32     }
33 };
34 #endif
35
36
37 **Explanation:
38 — **Line 9–11: The ‘push’ function adds an item to the top of the stack by appen
39 — **Line 13–15: The ‘pop’ function removes the top element from the stack by pop
40 — **Line 17–20: The ‘top’ function returns the last element (top of the stack).
41 — **Line 22–23: The ‘empty’ function checks if the stack is empty.

```

3 Queue Implementation (Queue.h)

Listing 2: Queue.h: Custom Queue Implementation

```

1 #ifndef QUEUE_H
2 #define QUEUE_H
3
4 #include <iostream>
5 #include <deque>
6
7 template<typename T>
8 class Queue {
9     private:
10         std::deque<T> elements;
11
12     public:
13         void enqueue(const T& item) {
14             elements.push_back(item);
15         }
16
17         void dequeue() {
18             if (!elements.empty()) {
19                 elements.pop_front();
20             }

```

```

21     }
22
23     T front() const {
24         if (!elements.empty()) {
25             return elements.front();
26         }
27         throw std::out_of_range("Queue is empty");
28     }
29
30     bool empty() const {
31         return elements.empty();
32     }
33 };
34 #endif
35
36
37 **Explanation:**
38 — **Line 9–11: The ‘enqueue’ function adds an item to the back of the queue by p
39 — **Line 13–15: The ‘dequeue’ function removes the front element by popping it fr
40 — **Line 17–20: The ‘front’ function returns the first element in the queue. If
41 — **Line 22–23: The ‘empty’ function checks if the queue is empty.

```

4 Palindrome Detection

The palindrome detection checks whether a string is the same forward and backward, ignoring spaces and capitalization. The following code was used:

Listing 3: Palindrome Detection Implementation

```

1  bool isPalindrome(const std::string& str) {
2      std::stack<char> s;
3      std::queue<char> q;
4
5      // Clean up the string: ignore spaces and case
6      for (char ch : str) {
7          if (isalpha(ch)) {
8              char lowerCh = tolower(ch);
9              s.push(lowerCh);
10             q.push(lowerCh);
11         }
12     }
13
14     // Check if the stack and queue are equal
15     while (!s.empty()) {
16         if (s.top() != q.front()) {
17             return false;

```

```

18         }
19         s.pop();
20         q.pop();
21     }
22     return true;
23 }
24
25
26 **Explanation:
27 - **Line 2–3: We initialize a stack and a queue. The stack will help reverse the
28 - **Line 6–10: Non-alphabetic characters are ignored, and alphabetic characters
29 - **Line 13–20: We compare characters from the top of the stack (last-in, first-out)

```

5 Sorting Algorithms

5.1 Selection Sort

Selection sort works by finding the smallest element from the unsorted part of the array and swapping it with the first unsorted element. Here's the implementation:

Listing 4: Selection Sort Implementation

```

1 int selectionSort(std::vector<std::string>& arr) {
2     int comparisons = 0;
3     std::size_t n = arr.size();
4
5     for (std::size_t i = 0; i < n - 1; ++i) {
6         std::size_t minIndex = i;
7         for (std::size_t j = i + 1; j < n; ++j) {
8             comparisons++;
9             if (arr[j] < arr[minIndex]) {
10                 minIndex = j;
11             }
12         }
13         std::swap(arr[i], arr[minIndex]);
14     }
15     return comparisons;
16 }
17
18
19 **Explanation:**
20 - **Line 4: The outer loop goes through the array, leaving one element unsorted
21 - **Line 6: The inner loop finds the smallest element.
22 - **Line 10: The elements are swapped, ensuring the smallest element is placed in

```

Selection sort is $\mathcal{O}(n^2)$ in both average and worst-case time complexity, since it always performs the same number of comparisons regardless of the initial ordering.

5.2 Insertion Sort

Insertion sort builds the sorted array one item at a time. The algorithm inserts each new element into the correct position within the sorted part of the array:

Listing 5: Insertion Sort Implementation

```

1  int insertionSort(std::vector<std::string>& arr) {
2      int comparisons = 0;
3      std::size_t n = arr.size();
4
5      for (std::size_t i = 1; i < n; ++i) {
6          std::string key = arr[i];
7          std::size_t j = i;
8          while (j > 0 && arr[j - 1] > key) {
9              arr[j] = arr[j - 1];
10             j--;
11             comparisons++;
12         }
13         arr[j] = key;
14         comparisons++;
15     }
16     return comparisons;
17 }
18
19 **Explanation:**
20 – **Line 4: The loop iterates through the array starting from the second element
21 – **Line 7–9: The current element is compared to the sorted part of the array and
```

Insertion sort is $\mathcal{O}(n^2)$ in the worst case but $\mathcal{O}(n)$ in the best case when the array is nearly sorted.

5.3 Merge Sort

Listing 6: Merge Sort Implementation

```

1  int merge(std::vector<std::string>& arr, std::vector<std::string>& temp, std::si
2      std::size_t i = left, j = mid + 1, k = left;
3      int comparisons = 0;
4
5      while (i <= mid && j <= right) {
6          comparisons++;
7          if (arr[i] <= arr[j]) {
8              temp[k++] = arr[i++];
```

```

9         } else {
10             temp[k++] = arr[j++];
11         }
12     }
13
14     while (i <= mid) {
15         temp[k++] = arr[i++];
16     }
17
18     while (j <= right) {
19         temp[k++] = arr[j++];
20     }
21
22     for (std::size_t m = left; m <= right; m++) {
23         arr[m] = temp[m];
24     }
25
26     return comparisons;
27 }
28
29 int mergeSortHelper(std::vector<std::string>& arr, std::vector<std::string>& temp,
30 int comparisons = 0;
31
32     if (left < right) {
33         std::size_t mid = left + (right - left) / 2;
34         comparisons += mergeSortHelper(arr, temp, left, mid);
35         comparisons += mergeSortHelper(arr, temp, mid + 1, right);
36         comparisons += merge(arr, temp, left, mid, right);
37     }
38
39     return comparisons;
40 }
41
42 int mergeSort(std::vector<std::string>& arr) {
43     std::vector<std::string> temp(arr.size());
44     return mergeSortHelper(arr, temp, 0, arr.size() - 1);
45 }
46
47
48 **Explanation:**
49 – **Line 3–7: The ‘merge’ function merges two sorted halves of the array into a
50 – **Line 20–24: After the merge, the remaining elements from the left and right
51 – **Line 28: The ‘mergeSortHelper’ function recursively splits the array into tw
52 – **Line 34–38: The ‘mergeSort’ function initializes a temporary array and calls

```

Merge sort’s time complexity is $\mathcal{O}(n \log n)$ due to the logarithmic number of

recursive calls and the linear merging process.

5.4 Quick Sort

Listing 7: Quick Sort Implementation

```
1 // Partition function to partition the array around a pivot
2 int partition(std::vector<std::string>& arr, std::size_t low, std::size_t high,
3   std::string pivot = arr[high];
4   std::size_t i = low;
5
6   for (std::size_t j = low; j < high; j++) {
7       comparisons++;
8       if (arr[j] <= pivot) {
9           std::swap(arr[i], arr[j]);
10          i++;
11      }
12  }
13  std::swap(arr[i], arr[high]);
14  return i;
15 }
16
17 // Helper function to recursively sort the array
18 int quickSortHelper(std::vector<std::string>& arr, std::size_t low, std::size_t
19   int comparisons = 0;
20
21   if (low < high && high < arr.size()) {
22       std::size_t pi = partition(arr, low, high, comparisons);
23       comparisons += quickSortHelper(arr, low, pi > 0 ? pi - 1 : 0);
24       comparisons += quickSortHelper(arr, pi + 1, high);
25   }
26
27   return comparisons;
28 }
29
30
31 int quickSort(std::vector<std::string>& arr) {
32     return quickSortHelper(arr, 0, arr.size() - 1);
33 }
34
35
36 **Explanation:**
37 — **Line 3–8: The ‘partition’ function selects a pivot element, partitions the a
38 — **Line 11–17: The ‘quickSortHelper’ function recursively applies Quick Sort on
39 — **Line 19–22: The ‘quickSort’ function initiates the sorting process by calling
   **Explanation:** Quick sort has a average time complexity of  $\mathcal{O}(n \log n)$ .
```

5.5 Main Program

The main program reads a list of magic items from a file, checks for palindromes, and applies different sorting algorithms (Selection Sort, Insertion Sort, Merge Sort, and Quick Sort) to the data. For each sorting algorithm, the number of comparisons made is outputted.

Listing 8: Main Program

```
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <string>
5 #include "Palindrome.h"
6 #include "SortingAlgorithms.h"
7
8 int main() {
9     std::ifstream inputFile("magicitems.txt");
10    if (!inputFile) {
11        std::cerr << "Error: -Unable-to-open-file-magicitems.txt\n";
12        return 1;
13    }
14
15    std::vector<std::string> magicItems;
16    std::string line;
17
18    // Read each line from the input file into the vector
19    while (std::getline(inputFile, line)) {
20        magicItems.push_back(line);
21    }
22    inputFile.close();
23
24    // Check each line for palindrome and output if found
25    std::cout << "Palindromes-in-the-file:\n";
26    for (const std::string& item : magicItems) {
27        if (isPalindrome(item)) {
28            std::cout << item << '\n';
29        }
30    }
31
32    // Step 3: Perform sorting and count comparisons for each algorithm
33    std::vector<std::string> itemsToSort;
34
35    // Selection Sort
36    itemsToSort = magicItems; // Copy the original list
37    knuthShuffle(itemsToSort); // Shuffle the list before sorting
38    int selectionComparisons = selectionSort(itemsToSort);
```

```

39     std::cout << "Selection-Sort-comparisons:-" << selectionComparisons << '\n';
40
41     // Insertion Sort
42     itemsToSort = magicItems; // Reset the list to original
43     knuthShuffle(itemsToSort); // Shuffle the list before sorting
44     int insertionComparisons = insertionSort(itemsToSort);
45     std::cout << "Insertion-Sort-comparisons:-" << insertionComparisons << '\n';
46
47     // Merge Sort
48     itemsToSort = magicItems; // Reset the list to original
49     knuthShuffle(itemsToSort); // Shuffle the list before sorting
50     int mergeComparisons = mergeSort(itemsToSort);
51     std::cout << "Merge-Sort-comparisons:-" << mergeComparisons << '\n';
52
53     // Quick Sort
54     itemsToSort = magicItems; // Reset the list to original
55     knuthShuffle(itemsToSort); // Shuffle the list before sorting
56     int quickComparisons = quickSort(itemsToSort);
57     std::cout << "Quick-Sort-comparisons:-" << quickComparisons << '\n';
58
59     return 0;
60 }
61
62 **Explanation:**
63 – **Line 6: We open the file ‘magicitems.txt’ which contains a list of magic iter
64 – **Line 12–16: The file contents are read line by line and stored in a ‘std::ve
65 – **Line 19–24: The code checks each item from the file to see if it is a palindi
66 – **Lines 29–50: For each sorting algorithm (Selection Sort, Insertion Sort, Mer
67 – A copy of the original ‘magicItems’ list is made.
68 – The list is shuffled using the ‘knuthShuffle()’ function to ensure the data
69 – The sorting algorithm is applied, and the number of comparisons made during
70 – The number of comparisons is then printed to the console.

```

6 Performance Comparison

Multiple test runs were conducted. The average number of comparisons for each sort is shown below:

Sorting Algorithm	Average Comparisons
Selection Sort	221445
Insertion Sort	111005
Merge Sort	5428
Quick Sort	6178

7 Conclusion

This document mainly presented the implementations and performance comparison of several sorting algorithms, with an included palindrome checking system. Merge sort ultimately performed the best with the least number of comparisons needed for every successful sort.